Project Report — Museum Ticketing & AI Cultural Guide

Project: Museum Ticketing & AI Cultural Guide
Author: Alessio Manera (Student ID: 905639)
Group: Group-11 (single-person group)
Course: Lab of Web Technologies (AY 2025–26), Prof. Zeynep Yucel — Ca' Foscari University of Venice
Submission date: September 4, 2026
Repository: `https://github.com/alessiomanera/web-tech-lab-2026`

---

1. Description and Goals

The problem

Booking a cultural visit in Italy is fragmented and high-friction. Ticketing is spread across dozens of venue-specific portals with inconsistent flows; deciding *what* to see — given a limited time budget, a mixed-age group, or a specific interest such as Renaissance painting or Ancient Rome — is left entirely to the visitor and a pile of browser tabs. There is no single place that both sells the ticket and helps plan the visit.

The solution

This project is a full-stack web application that does both. It presents a curated catalog of 12 bookable cultural experiences across six Italian cities (Florence, Rome, Venice, Milan, Turin, Naples), backed by 12 museums and cultural sites, one per experience, and pairs it with a conversational AI concierge that recommends experiences from that same catalog and can drop a ready-to-book card directly into the chat. Recommendations are grounded in the application's own relational database rather than the model's open-ended knowledge, so the assistant only ever suggests things that actually exist and are actually bookable.

The four modules

1. **User Authentication & Cultural Profiling** — registration and login with Flask server-side sessions and Werkzeug PBKDF2 password hashing; an account dashboard listing digital passes and past visits; a persisted Markdown "Cultural Taste Profile" that the concierge updates as the conversation reveals preferences.
2. **Experience & Museum Catalog Directory** — a dynamic catalog of experiences and museums with real-time relational filtering by city, theme, and free-text keyword search, plus per-experience detail pages covering duration, pricing, inclusions, and highlights.
3. **4-Step Ticketing & Reservation Engine** — a frictionless booking wizard: choose the package, pick add-ons, select a visit date and time slot, and receive an instant digital pass with a unique booking code. Post-visit, the user can leave a 1–5 star rating and a written review, gated to bookings they actually own.
4. **Grounded AI Cultural Concierge** — the core feature of the project. Conversational discovery powered by the Google Gemini API using Retrieval-Augmented Generation (RAG): the live `experiences` catalog is injected into the model's system instruction so it recommends only real, bookable packages, it emits an in-band `[RECOMMEND: id=…, title=…, city=…, price=…]` protocol tag that the front end parses into a real bookable card, and it writes back a structured Markdown taste profile that persists across sessions. A

> Gemini API key is required to run this path. **Without a key** (or if an API call fails mid-session), the concierge falls back to a deliberately limited keyword-matching mode: it matches the message against catalog cities and themes and returns a single templated recommendation card. This fallback does **not** perform RAG grounding, multi-factor reasoning, or taste-profile extraction — it exists purely as a network/quota safety net so the rest of the application stays navigable during evaluation, not as an equivalent of the real concierge.

---

2. How to Run the Application

Prerequisites

- Python 3.10 or newer
- Git — only needed if cloning; if you already have this project as a folder (e.g. unzipped from a Moodle submission), skip Git and start at "Create and activate a virtual environment" below, from inside that folder.
- An internet connection (to install dependencies, and optionally to reach the Gemini API).

Setup

```
git clone https://github.com/alessiomanera/web-tech-lab-2026.git
cd web-tech-lab-2026

# Create and activate a virtual environment
python -m venv venv
# Windows (PowerShell):   .\venv\Scripts\Activate.ps1
#   -> if PowerShell refuses with "running scripts is disabled on this system",
#      run this first, then retry the line above:
#      Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
# Windows (cmd):          .\venv\Scripts\activate.bat
# macOS / Linux:          source venv/bin/activate
# Your prompt should now start with "(venv)" - that confirms it worked.

pip install -r requirements.txt   # installs 3 packages: Flask, python-dotenv, google-generativeai

# Copy the environment template
cp .env.example .env        # Windows: copy .env.example .env

python seed.py               # create and populate the database - look for
                              # "Database successfully seeded..." to confirm it worked
python app.py                 # start the development server - look for
                               # "Running on http://127.0.0.1:5000"
```

Then open `http://127.0.0.1:5000/` in a browser. If that port is already taken by something else on your machine ("Address already in use" — common on macOS, where AirPlay Receiver defaults to port 5000), either free port 5000 or start the app on another one: `python -c "from app import create_app; create_app().run(port=5001)"`, then open `http://127.0.0.1:5001/` instead.

The two `.env` settings

`.env` (copied from `.env.example` above) holds two settings — worth being clear on both before assuming something is broken:

- `FLASK_SECRET_KEY` — signs login session cookies. **You do not need to change this to run or evaluate the project.** The placeholder value works fine locally; it only matters for a real internet-facing deployment, which this isn't.
- `GEMINI_API_KEY` — recommended for full evaluation, see below. Unlike the secret key, this one *does* change what you'll see if you leave it as the placeholder.

Gemini API key — recommended for full evaluation

The AI Cultural Concierge (Module 4) is the project's core feature, and it runs on the Google Gemini API. **To evaluate it properly, set a key:**

1. Go to Google AI Studio and sign in with any Google account.
2. Click **Create API key**.
3. Copy the key, and paste it into `.env` as `GEMINI_API_KEY=` (replacing `your_gemini_api_key_here`). Takes about 2 minutes, no credit card required.

Model defaults to `gemini-flash-lite-latest`, overridable via `GEMINI_MODEL`. With a key, the concierge does RAG grounding on the live catalog and extracts a persistent taste profile from the conversation.

Without a key the app still starts and every other module works, but the concierge drops to a labelled *offline demo mode*: a deterministic keyword matcher that returns one templated recommendation card and performs **no** RAG grounding and **no** taste-profile updates. The concierge view shows a banner when this mode is active. The fallback is a deliberate resilience feature (network/quota safety net), not a substitute for the real concierge.

Demo account

`seed.py` creates a ready-to-use account with a pre-populated Cultural Taste Profile, so the booking flow, concierge, and dashboard are all populated on first login:

Email	Password
`alessio@example.com`	`password123`

You may also register a fresh account at `/register`.

> **Warning:** `seed.py` **drops and recreates** every table. Any accounts or bookings created since the last seed are permanently deleted. Run it once on first setup, or whenever a clean demo state is wanted.

---

3. Contributions and Roles

This is a **single-person group** (Group-11). Every decision behind this project — the competitor research, the architecture and technology choices (including moving off Flask-SQLAlchemy to raw `sqlite3`), the Neubrutalism design system, the test suite, the documentation — was made and owned by the sole author, Alessio Manera. There are no other people involved.

For clarity, the work breaks down by workstream as follows, all performed by the same person:

Workstream	Concrete artifacts produced
Database & schema design	`schema.sql` (relational DDL: `users`, `museums`, `exhibitions`, `experiences`, `tickets`; `ON DELETE CASCADE` on tickets, `ON DELETE SET NULL` on `experiences.museum_id`); `database.py` (per-request `sqlite3` connection context manager with `PRAGMA foreign_keys = ON`); `seed.py` (12 museums, 12 curated experiences across 6 cities, demo account).
Backend & REST API	`app.py` (application factory, 400/404/500 handlers); `routes.py` (page routes + `/api/book`, `/api/chat`, `/api/feedback`, `/api/profile/reset-memory`); `auth.py` (registration, login, logout, `@login_required`, safe `next` redirect handling). 100% parameterized SQL throughout.
AI / RAG integration	`_call_gemini_concierge()` in `routes.py`: RAG prompt construction grounded on the live catalog, the `[RECOMMEND: …]` in-band protocol, Markdown taste-profile extraction and persistence, and the local heuristic fallback matcher.
Frontend & design system	13 Jinja2 templates extending `base.html`; the vanilla Neubrutalist CSS system (`variables.css`, `layout.css`, `components.css`, `utilities.css`); six vanilla JavaScript files (`main.js`, `api.js`, `ui.js`, `bookingWizard.js`, `concierge.js`, `profile.js`) loaded with `defer` in dependency order from `base.html` — deliberately classic scripts sharing a global scope rather than ES modules, since the app ships without a bundler or build step; the anti-FOUC theme bootstrap and the light/dark toggle.
QA & test suite	`tests/` — 61 `unittest` integration tests against isolated temporary SQLite databases, covering database constraints, authentication, catalog filtering, booking arithmetic, concierge RAG and fallback, the feedback loop and its validation, and the custom error pages.
Documentation	`README.md`, `DOCS/1-Page_Project_Proposal.md`, `DOCS/Competitor_Analysis.md`, `ROADMAP.md`, `LICENSE`, and this report.

Because there is no team to divide work across, the grading dimension of *collaboration and group structure* is satisfied here by demonstrating **complete, legible, individual authorship** of a coherent full-stack system, with a development history (75+ commits) that evidences incremental work.

---

4. Architecture Overview

The application uses the Flask **application-factory pattern**. `create_app(test_config=None)` builds and configures the app; passing a `test_config` lets the test suite point each test at its own throwaway SQLite file, so tests never touch a shared database.

Request lifecycle:

```
Browser
  │  HTTP request
  ▼
Flask routing  →  Blueprint view  (main_bp in routes.py  /  auth_bp in auth.py)
  │
  ▼
get_db()  →  per-request sqlite3 connection  (PRAGMA foreign_keys = ON, Row factory)
  │
  ▼
Parameterized SQL  ( ? placeholders only — no string interpolation anywhere )
  │
  ├─►  Jinja2 template render  →  HTML response   (page routes)
  └─►  jsonify(...)            →  JSON response   (/api/* routes consumed by fetch)
```

- **No ORM.** All persistence is raw `sqlite3` with parameterized queries — a deliberate choice to keep the data layer transparent and auditable, and consistent with the database pattern demonstrated in the course's own `week_5_database_exercise` (raw `sqlite3`, `Row` factory, hand-written SQL, Werkzeug hashing).
- **Blueprints** separate authentication (`auth_bp`) from the core application (`main_bp`).
- **Sessions** are Flask's signed cookies; `@login_required` gates the booking wizard, concierge, and profile.
- **Error handling** is centralised: `abort(404)` and unhandled errors render branded `400.html` / `404.html` / `500.html` pages.

---

5. Testing

```
python -m unittest discover -s tests -p "test_*.py" -v
```

61 tests, all passing. They run at integration level (real Flask test client, real SQLite) against an isolated temporary database created and destroyed per test.

Module	What it proves
`test_database.py`	`PRAGMA foreign_keys = ON` is active per connection; `UNIQUE` constraints on email and booking code; `ON DELETE CASCADE` / `SET NULL` behave as declared.
`test_auth.py`	Registration, Werkzeug PBKDF2 hashing (password never stored in clear), login success/failure, session cleared on logout, `@login_required` redirects, safe vs. rejected `?next=` redirect targets (open-redirect protection), and CSRF enforcement — a POST with a missing or forged token is rejected on both the HTML forms and the JSON API, GET requests are unaffected, and the same request succeeds once the token is presented.
`test_catalog.py`	City / theme / keyword filtering produce the correct subset; detail pages render; a missing experience id renders the branded custom 404 page.
`test_booking.py`	4-step wizard rendering, visit-date bounds, time-slot validation, guest-count bounds (1–6), add-on pricing arithmetic, and that add-on prices are taken from the catalog rather than the request payload.

Module	What it proves
`test_concierge.py`	Grounded Gemini RAG (mocked) parses `[RECOMMEND: …]` tags and persists the extracted taste profile; the offline heuristic fallback matches by city and by theme; empty messages are rejected; a stored taste profile containing markup is escaped, never rendered as HTML.
`test_feedback.py`	Ratings persist; reviews are gated to the owning user (404 otherwise); non-numeric ratings return 400 (not 500); ratings outside 1–5 are rejected.
`test_errors.py`	Custom `400.html` / `404.html` / `500.html` pages render with the correct status codes; the profile dashboard renders.

---

6. Security Measures

Measure	Implementation
SQL injection	100% parameterized queries (`?` placeholders) — verified across every statement in `routes.py` , `auth.py` , `database.py` , `seed.py` . No f-string or `%` -formatted SQL anywhere.
Password storage	Werkzeug `generate_password_hash` / `check_password_hash` (PBKDF2-SHA256, per-user salt). Plaintext passwords are never stored or logged.
Authentication & authorisation	Flask signed-cookie sessions; `@login_required` on the booking wizard, concierge, and profile; feedback writes verify `user_id` ownership of the target booking.
Server-side output escaping	Jinja2 autoescaping on all templates. The one former `safe` filter on user-controlled taste-profile text was removed and replaced with plain interpolation plus a CSS `white-space: pre-line` class.
Client-side output escaping	`concierge.js` escapes every HTML-significant character in user chat input, model output, and the stored taste profile before it reaches the DOM (`escapeHtml` ; taste profile written via `textContent` , not `innerHTML`). This closes a reflected/stored XSS vector in the chat renderer.
Cross-site request forgery (CSRF)	Every state-changing request must present a per-session token minted with `secrets.token_urlsafe(32)` and compared with `secrets.compare_digest` . It reaches the server as a hidden `csrf_token` field on the HTML forms and an `X-CSRFToken` header on the JSON endpoints, enforced centrally by a `before_request` hook rather than per-route. Implemented directly rather than via Flask-WTF, so the mechanism is visible in the codebase. Second layer: the session cookie is set `HttpOnly` and `SameSite=Lax` explicitly, so the browser will not attach it to a cross-site POST in the first place.
Open-redirect protection	The post-login `?next=` target is accepted only if it is a same-site relative path; absolute URLs and protocol-relative `//` targets are discarded and the user is sent to the home page.
Server-side pricing integrity	A booking request may only name *which* add-ons it wants. `create_booking()` re-reads each add-on's name and price from the chosen experience's own catalog entry, drops unknown ids and collapses duplicates, so a crafted payload cannot invent an add-on or discount the total. Covered by two regression tests.

Measure	Implementation
Booking-code collisions	Booking codes are `EXP-2026-` + 6 random alphanumerics on a `UNIQUE` column; insertion retries on the rare `IntegrityError` instead of surfacing a 500.

---

7. Known Limitations

Honest scope boundaries of the delivered system:

- **Session-scoped CSRF tokens.** The token lives in the signed session cookie and lasts as long as the session, rather than rotating per form. That is the standard trade-off for a server-rendered app of this size; per-request rotation would break the back button and concurrent tabs without meaningfully raising the bar here.
- **The concierge is single-turn.** No running conversation history is sent to Gemini; continuity between messages is carried only by the persisted Cultural Taste Profile.
- **RAG grounding covers the `experiences` table only** — the 12 bookable packages. It does not ground on `museums` or `exhibitions`, and does not know venue logistics such as street addresses, general opening hours, or physical accessibility.
- **`exhibitions` is schema-only.** The table exists in `schema.sql`, is populated by `seed.py`, and enforces a foreign key to `museums`, but no route or template currently reads from it.
- **No payment integration.** Booking issues a digital pass and a booking code; no money changes hands.
- **No capacity or inventory model.** Time slots never sell out; there is no per-slot seat count.
- **Not a production deployment.** SQLite and the Flask development server are appropriate for the assignment and local evaluation, not for production traffic.
- **Responsive coverage** is verified at 320, 375, 768 and 1440 px across every page (no horizontal overflow at any width), but the layout is designed desktop-first; a phone gets a correct stacked layout rather than a purpose-built mobile experience.